MongoDB CDC with Debezium, Kafka, NiFi

Debezium is an open-source Change Data Capture (CDC) platform that uses MongoDB Change Streams to capture every insert, update, and delete event. In Debezium 2.x, the connector exclusively uses the Change Streams API (which internally builds on the oplog) to stream database changes to Kafka topics for event-driven architectures.

1. MongoDB side

Debezium reads from the oplog through change streams, which only exist on a replica set.

On the primary:

```
rs.initiate({
  _id: "rs0",
  members: [{ _id: 0, host: "<mongo-host>:<port>" }]
})
```

Add a user for Debezium:

```
db.createUser({
  user: "MDB_CDC",
  pwd: "<password>",
  roles: [
    { role: "read", db: "mydb" },
    { role: "clusterMonitor", db: "admin" },
    { role: "read", db: "local" }
  ]
})
```

- `clusterMonitor` — needed to read replica set status/oplog metadata.
- `read` on `local` — needed because the oplog itself lives in the `local` database.

2. Install the connector on Kafka Connect

Drop the Debezium MongoDB connector JARs into your Connect plugin path (e.g. `<kafka-plugins-path>/debezium-connector-mongodb/`), then restart the Connect workers so they pick it up.

Register the connector via the REST API:

```
curl -X POST http://connect-host:8083/connectors \
  -H "Content-Type: application/json" \
  -d @mongo-connector.json
```

`mongo-connector.json`:

```
{
  "name": "mongo-cdc-connector",
  "config": {
    "connector.class": "io.debezium.connector.mongodb.MongoDbConnector",
    "mongodb.connection.string": "mongodb://MDB_CDC:<password>@<mongo-
host1>:<port>,<mongo-host2>:<port>/?replicaSet=rs0",
    "topic.prefix": "dbserver1",
    "collection.include.list": "mydb.orders,mydb.customers",
    "snapshot.mode": "initial",
    "capture.mode": "change_streams_update_full",
    "tombstones.on.delete": "true",
    "key.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "key.converter.schemas.enable": "false",
    "value.converter.schemas.enable": "false"
  }
}
```

What each parameter controls:

- `mongodb.connection.string` — points to all replica set members, not just one, so Debezium can follow failover.
- `topic.prefix` — prefix used to name the Kafka topics it creates, one per collection: `dbserver1.mydb.orders`.
- `collection.include.list` — whitelist of collections to capture. Anything not listed is ignored, keeps noise down.
- `snapshot.mode` — `initial` copies existing data first then streams new changes; use `no_data` if you only care about changes going forward, `never` if you're resuming and trust existing state.
- `capture.mode` — `change_streams_update_full` makes Mongo return the complete updated document on updates instead of a diff. Without this you only get changed fields, which is harder to apply downstream.
- `tombstones.on.delete` — emits a null-value message after a delete, used for Kafka log compaction. You'll filter these out in NiFi.

Check it came up:

```
curl http://connect-host:8083/connectors/mongo-cdc-connector/status
```

3. Event shape

Each message on the topic:

```
{
  "before": null,
```

```
  "after": "{\"_id\": {\"$oid\": \"...\"}, \"status\": \"shipped\",
\"amount\": 250}",
  "op": "u",
  "ts_ms": 1752480000123
}
```

`op` tells you what happened:

- `r` — snapshot read (initial load, treat as insert)
- `c` — insert
- `u` — update
- `d` — delete

`after` is a JSON string, not a nested object — parse it twice.

4. NiFi flow

```
ConsumeKafka
   → EvaluateJsonPath (pull op, after, before as attributes)
   → RouteOnAttribute (branch on ${op})
       ├─ c / r → parse "after" → PutDatabaseRecord (INSERT)
       ├─ u     → parse "after" → PutDatabaseRecord (UPDATE / upsert by
_id)
       └─ d     → parse "before" for _id → PutDatabaseRecord (DELETE by
_id)
```

ConsumeKafka

- Kafka Brokers: your broker list
- Topic Name(s): `dbserver1.mydb.orders`, or a regex `dbserver1\.mydb\..*` to grab every captured collection in one processor
- Group ID: a name unique to this flow, so offsets are tracked separately from other consumers
- Offset Reset: `earliest` on first run, then NiFi manages committed offsets itself

EvaluateJsonPath Extract before you branch:

- `op` → `$.op`
- `after_raw` → `$.after`
- `before_raw` → `$.before`

RouteOnAttribute

- insert/read: `${op:equals('c')}` or `${op:equals('r')}`
- update: `${op:equals('u')}`
- delete: `${op:equals('d')}`
- drop tombstones: add a rule that discards flowfiles where `after_raw` and `before_raw` are both empty

Insert / update branch `after_raw` holds the document as a string. Replace flowfile content with its parsed value (EvaluateJsonPath with destination set to flowfile-content, or a JSON reader/writer pair), then send to `PutDatabaseRecord`. For updates, configure the statement type as UPDATE, keyed on `_id`, so it acts as an upsert if you also allow insert-on-no-match.

Delete branch Pull `_id` out of `before_raw` (or out of `after`'s `documentKey` if using full-document mode), then run `PutDatabaseRecord` with statement type DELETE, matched on `_id`.

5. Test it

```
db.orders.insertOne({ status: "pending", amount: 100 })
db.orders.updateOne({ status: "pending" }, { $set: { status: "shipped" } })
db.orders.deleteOne({ status: "shipped" })
```

Watch the topic to confirm the three events show up with `op` = c, u, d, then check the target table gets the row inserted, updated, and removed in order. Since `_id` is used as the Kafka message key, all three events for that document land in the same partition, so order is preserved.

Notes

- If Connect restarts, it resumes from the committed offset in Kafka — no replay from the start, no gaps, as long as the oplog hasn't rolled past what wasn't yet consumed.
- Mongo has no fixed schema, so don't hardcode fields in NiFi's record readers — use schema inference or a registry, otherwise a new field in a document breaks the flow.
- Set backpressure thresholds on the queue after ConsumeKafka so a slow database sink doesn't pile up unbounded memory in NiFi.